

Week 9: Convolutional Neural Networks

Deep Learning – IITM BS
Comprehensive Notes + Practice Problems

Contents

1	Motivation: Why CNNs?	2
1.1	The Problem with Fully Connected Networks for Images	2
1.2	Key Ideas Behind CNNs	2
2	The Convolution Operation	2
2.1	1D Convolution	2
2.2	2D Convolution (Cross-Correlation)	2
2.3	Detailed Hand Computation Example	3
3	Output Size Formula	3
3.1	Same Padding	3
3.2	Multiple Filters and 3D Convolution	3
4	Parameter Counting	4
5	Pooling Layers	4
5.1	Types of Pooling	4
6	Backpropagation in CNNs	4
6.1	Through Pooling Layers	4
6.2	Through Conv Layers	4
7	CNN Architecture: Full Pipeline	5
8	Famous Architectures	5
8.1	ResNet: Skip Connections	5
9	Weight Sharing in CNNs	5
10	Practice Problems	6
11	Solutions	8

1 Motivation: Why CNNs?

1.1 The Problem with Fully Connected Networks for Images

- A $227 \times 227 \times 3$ image has **154,587** input neurons
- If first hidden layer has 1000 neurons: $154,587 \times 1000 \approx 155M$ parameters in *one* layer
- No spatial structure is exploited—pixel (0,0) treated same as pixel (100,100)
- Hubel & Wiesel (1962): neurons in visual cortex have **local receptive fields**

1.2 Key Ideas Behind CNNs

1. **Local connectivity**: Each neuron connects to a small region (receptive field)
2. **Weight sharing**: Same filter (kernel) slides across entire image
3. **Translation invariance**: Features detected regardless of position
4. **Hierarchical features**: Early layers detect edges, later layers detect complex patterns

2 The Convolution Operation

2.1 1D Convolution

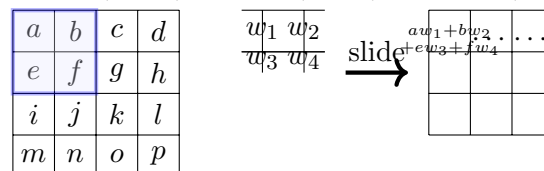
$$(x * w)(t) = \sum_{a=0}^{\infty} x(a) \cdot w(t-a)$$

2.2 2D Convolution (Cross-Correlation)

In practice, deep learning libraries implement **cross-correlation**, not true convolution:

$$g(i, j) = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I(i+a, j+b) \cdot K(a, b)$$

Input (4×4) **Kernel** (2×2) **Output** (3×3)



When are convolution and correlation the same?

When the kernel K is symmetric (i.e., $K = K^{\text{flipped}}$). In practice, since the kernel is learned, it doesn't matter which operation we use—the learned weights will compensate.

2.3 Detailed Hand Computation Example

Convolution by Hand

Input: $X = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$, Kernel: $W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, Stride = 1, No padding.

Output size = $1 + \frac{3-2}{1} = 2$, so output is 2×2 .

Position (0,0): Window = $\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$. Output = $2 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 3$

Position (0,1): Window = $\begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}$. Output = $1 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 2 \cdot 1 = 3$

Position (1,0): Window = $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Output = $0 \cdot 1 + 1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 = 0$

Position (1,1): Window = $\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$. Output = $1 \cdot 1 + 2 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 2$

Output = $\begin{bmatrix} 3 & 3 \\ 0 & 2 \end{bmatrix}$

3 Output Size Formula

THE Most Important CNN Formula

$$W_{\text{out}} = 1 + \frac{W_{\text{in}} - F + 2P}{S}$$

W_{in} = input spatial dimension (width or height)

F = filter/kernel size

P = padding (number of zeros added on each side)

S = stride

The output depth (number of channels) = K = number of filters.

3.1 Same Padding

To make output size = input size (when stride = 1):

$$W_{\text{in}} = 1 + \frac{W_{\text{in}} - F + 2P}{1} \implies P = \frac{F - 1}{2}$$

Exam Trap: Same Padding with Stride > 1

“Same padding” with stride > 1 does **NOT** preserve spatial dimensions! The formula $P = (F - 1)/2$ only gives output = input when stride = 1.

3.2 Multiple Filters and 3D Convolution

For input of size $W_1 \times H_1 \times D_1$ with K filters of size $F \times F \times D_1$:

- Each filter *spans the full depth* D_1 of the input
- Each filter produces a *single* 2D feature map
- K filters produce output depth = K

Output dimensions: $W_2 \times H_2 \times D_2$ where

$$W_2 = 1 + \frac{W_1 - F + 2P}{S}, \quad H_2 = 1 + \frac{H_1 - F + 2P}{S}, \quad D_2 = K$$

4 Parameter Counting

Conv Layer Parameters

Each filter has size $F \times F \times D_{\text{in}}$ (note: includes input depth!).

With bias: $\underbrace{F \times F \times D_{\text{in}} \times K}_{\text{weights}} + \underbrace{K}_{\text{biases}}$

Without bias: $F \times F \times D_{\text{in}} \times K$

Pooling layers: ZERO learnable parameters!

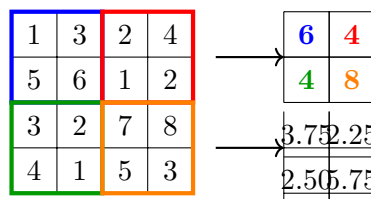
Common Exam Mistake

Forgetting to include D_{in} (input depth) in parameter counting. A 3×3 filter on an RGB image is actually $3 \times 3 \times 3 = 27$ weights per filter, not 9!

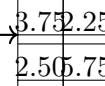
5 Pooling Layers

5.1 Types of Pooling

Input (4×4) MaxPool (2×2 , S=2)



AvgPool (2×2 , S=2)



- **Max Pooling:** Takes maximum value in each window
- **Average Pooling:** Takes mean of values in each window
- Same output size formula applies (with pool size as F , pool stride as S)
- Pooling does NOT change depth (number of channels)

6 Backpropagation in CNNs

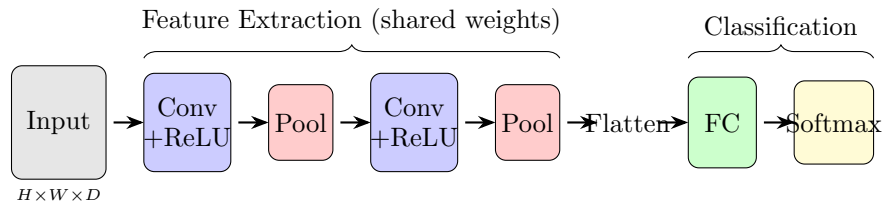
6.1 Through Pooling Layers

- **Max Pooling:** Gradient flows *only* to the position of the max value. All other positions get gradient 0.
- **Average Pooling:** Gradient is distributed *equally* to all positions in the window (each gets $\frac{1}{F^2}$ of the upstream gradient).

6.2 Through Conv Layers

- **Weight gradient** ($\frac{\partial L}{\partial W}$): Convolve input X with the upstream gradient
- **Input gradient** ($\frac{\partial L}{\partial X}$): Convolve the flipped weights W with the upstream gradient (full convolution)

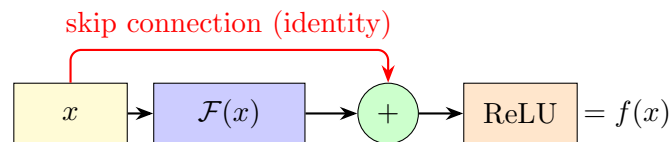
7 CNN Architecture: Full Pipeline



8 Famous Architectures

Network	Year	Key Innovation	Params
AlexNet	2012	ReLU, dropout, 227×227 input	~27M
ZFNet	2013	Better hyperparams than AlexNet	similar
VGGNet	2014	All 3×3 kernels, very deep (16–19 layers)	~138M
GoogLeNet	2014	Inception module (multi-size filters)	~5M
ResNet	2015	Skip connections (residual learning)	varies

8.1 ResNet: Skip Connections



$$f(x) = \text{ReLU}(x + \mathcal{F}(x))$$

If $\mathcal{F}(x) \approx 0$, then $f(x) \approx x$. The network can “at worst” learn the identity.

9 Weight Sharing in CNNs

- The **same filter** slides across all positions of the input
- This means the same weights detect the same feature everywhere
- Dramatically reduces parameter count vs. fully connected
- This is why sequence length / image size doesn’t affect parameter count in conv layers

10 Practice Problems

Problem 1: Output Size Calculation [2 marks]

A 6×6 grayscale image. Apply convolution with 4 filters of size 3×3 , stride 2, padding 1. What is the output shape?

Problem 2: Parameter Counting [2 marks]

A 5×5 image passes through: Conv(3×3 , stride 1, no padding, 1 filter, no bias) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2 , stride 2). How many learnable parameters total?

Problem 3: Convolution by Hand [3 marks]

Compute convolution (stride 1, no padding):

$$X = \begin{bmatrix} 1 & -3 & 0 \\ 0 & 1 & -2 \\ 1 & 0 & 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Problem 4: Same Padding [2 marks]

Image size $L \times L$, filter size $F \times F$, stride = 1. Derive the padding P needed so that output size equals input size.

Problem 5: Full Pipeline [3 marks]

Input: $8 \times 8 \times 3$ image.

Layer 1: Conv(3×3 , 16 filters, stride 1, padding 1, bias)

Layer 2: MaxPool(2×2 , stride 2)

Layer 3: Conv(3×3 , 32 filters, stride 1, no padding, bias)

Compute: (a) output size after each layer, (b) total parameters.

Problem 6: CNN Backprop Gradient [3 marks]

A 5×5 image X is convolved with $W = \begin{bmatrix} 1 & -3 & 0 \\ 0 & 1 & -2 \\ 1 & 0 & 2 \end{bmatrix}$ (stride 1, no padding, no bias, 1 filter).

After ReLU and MaxPool(2×2 , stride 2), the output $\hat{y}$ is a scalar. Loss: $L = \frac{1}{2}(\hat{y} - y)^2$.

Given:

$$X = \begin{bmatrix} 2 & 1 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 \\ 0 & 1 & 2 & 3 & 2 \\ 1 & 0 & 1 & 2 & 1 \\ 2 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$y = 1$.

(a) Compute the convolution output (before ReLU).

(b) Apply ReLU.

(c) Apply MaxPool.

(d) Find $\frac{\partial L}{\partial W_{12}}$ (the weight at row 1, col 2 of the kernel).

Problem 7: ResNet [3 marks]

ResNet with two residual blocks: $f(x) = x + \text{ReLU}(w \cdot x)$.

Block 1: $w_1 = 1$, Block 2: $w_2 = -5$.

Input $x = 1$. Compute the final output.

Problem 8: Multi-Layer Param Count [3 marks]

A CNN has:

Layer 1: Conv 5×5 , 6 filters, input is $28 \times 28 \times 1$, with bias

Layer 2: AvgPool 2×2 , stride 2

Layer 3: Conv 5×5 , 16 filters, with bias

Layer 4: AvgPool 2×2 , stride 2

Compute: (a) output dimensions after each layer, (b) total parameters.

11 Solutions

Problem 1: $W_{\text{out}} = 1 + \frac{6-3+2(1)}{2} = 1 + \frac{5}{2} = 1 + 2 = 3$. Output: $\mathbf{3 \times 3 \times 4}$.

Problem 2: Conv kernel: $3 \times 3 = 9$. No bias. Pool: 0. Total = **9**.

Problem 3: Output size = $1 + (3 - 2)/1 = 2$, so 2×2 .

$$(0, 0): 1(1) + (-3)(0) + 0(0) + 1(-1) = 1 - 1 = 0$$

$$(0, 1): (-3)(1) + 0(0) + 1(0) + (-2)(-1) = -3 + 2 = -1$$

$$(1, 0): 0(1) + 1(0) + 1(0) + 0(-1) = 0$$

$$(1, 1): 1(1) + (-2)(0) + 0(0) + 2(-1) = 1 - 2 = -1$$

$$\text{Output} = \begin{bmatrix} 0 & -1 \\ 0 & -1 \end{bmatrix}$$

Problem 4: Set $W_{\text{out}} = W_{\text{in}}$: $W = 1 + (W - F + 2P)/1 \implies W - 1 = W - F + 2P \implies 2P = F - 1 \implies \boxed{P = (F - 1)/2}$.

Problem 5:

(a) Layer 1: $1 + (8 - 3 + 2)/1 = 8$. Output: $8 \times 8 \times 16$.

Layer 2: $1 + (8 - 2)/2 = 4$. Output: $4 \times 4 \times 16$.

Layer 3: $1 + (4 - 3 + 0)/1 = 2$. Output: $2 \times 2 \times 32$.

(b) L1: $3 \times 3 \times 3 \times 16 + 16 = 448$. L2: 0. L3: $3 \times 3 \times 16 \times 32 + 32 = 4640$. Total = **5088**.

Problem 6: Conv output is 3×3 . Computing each position:

$$(0, 0): 2(1) + 1(-3) + 0(0) + 3(0) + 2(1) + 1(-2) + 0(1) + 1(0) + 2(2) = 2 - 3 + 0 + 0 + 2 - 2 + 0 + 0 + 4 = 3$$

(Similarly compute all 9 positions, apply ReLU, then MaxPool $\rightarrow 1 \times 1$ output.)

The gradient $\frac{\partial L}{\partial W_{12}}$: trace back through MaxPool (only the max position contributes), then through ReLU (zero if pre-activation was negative), then multiply by the corresponding input patch value.

Problem 7: Block 1: $f(1) = 1 + \text{ReLU}(1 \times 1) = 1 + 1 = 2$.

Block 2: $f(2) = 2 + \text{ReLU}(-5 \times 2) = 2 + \text{ReLU}(-10) = 2 + 0 = \mathbf{2}$.

Problem 8:

(a) L1: $1 + (28 - 5)/1 = 24$. Out: $24 \times 24 \times 6$. L2: $24/2 = 12$. Out: $12 \times 12 \times 6$.

L3: $1 + (12 - 5)/1 = 8$. Out: $8 \times 8 \times 16$. L4: $8/2 = 4$. Out: $4 \times 4 \times 16$.

(b) L1: $5 \times 5 \times 1 \times 6 + 6 = 156$. L2: 0. L3: $5 \times 5 \times 6 \times 16 + 16 = 2416$. L4: 0. Total = **2572**.